

Programação Recursiva: processamento de listas em Lisp — Parte 3

Orlando de A Figueiredo

IGCE/Unesp

26 de Agosto de 2017

A técnica do parâmetro-acumulador

- ▶ Em muitos casos, programas recursivos admitem versões iterativas.
- ▶ Uma técnica muito empregada é usar parâmetros extras que servem para passar adiante os resultados parciais do programa.
- ▶ Isso tem impacto sobre o desempenho.

Exemplo: o fatorial

Duas versões da função fatorial.

```
(define (fat-rec n)
  (if (zero? n)
      1
      (* n (fat-rec (sub1 n)))))

(define (fat-iter n fat)
  (if (zero? n)
      fat
      (fat-iter (sub1 n) (* n fat))))
```

Execução da versão recursiva

```
(fat-rec 4) =  
(* 4 (fat-rec 3)) =  
(* 4 (* 3 (fat-rec 2))) =  
(* 4 (* 3 (* 2 (fat-rec 1)))) =  
(* 4 (* 3 (* 2 (* 1 (fat-rec 0))))) =  
(* 4 (* 3 (* 2 (* 1 1)))) =  
(* 4 (* 3 (* 2 1))) =  
(* 4 (* 3 2)) =  
(* 4 6) =  
24
```

Execução da versão iterativa

```
(fat-iter 4 1) =  
(fat-iter 3 (* 4 1)) =  
(fat-iter 2 (* 3 4)) =  
(fat-iter 1 (* 2 12)) =  
(fat-iter 0 (* 1 24)) =  
24
```

Desempenho

- ▶ *TCO — Tail Call Optimization* é uma técnica muito empregada em implementações de Lisp.
- ▶ Pode ser empregada quando uma função termina apenas retornando o resultado de uma chamada recursiva.
- ▶ Ou seja, a última coisa da função é fazer a chamada recursiva e retornar seu resultado.
- ▶ Em nível de linguagem de máquina, em vez de empilhar a chamada recursiva, faz-se um jump.

```
(fat-rec 100000000)
## 0 programa ficou sem memória !!
(fat-iter 100000000)
>>> 413452524523452... páginas e páginas
```

Programa exemplo: tamanho da lista (cmpr-aux)

- ▶ Adicione um parâmetro.
- ▶ No caso base da recursão, use o parâmetro (acumulador) como resultado.
- ▶ No caso geral, faça a chamada usando o parâmetro como lugar para fazer a conta.
- ▶ Ao usar a função, o valor conveniente deve ser usado para iniciar o acumulador.

```
(define (cmpr-aux l cmp)
  (if (null? l)
      cmp
      (cmpr-aux (cdr l) (add1 cmp))))

(cmpr-aux '(10 100 100) 0)
>>> 3
```

A fachada

- ▶ É muito inconveniente expor o usuário da função à digitação de um parâmetro extra com o valor inicial esperado de acumulador.
- ▶ Assim, costuma-se criar uma função de fachada, que omite o parâmetro inicial, e que chama a função iterativa com o valor inicial devido.

```
(define (cmpr2 1)
  (cmpr-aux 1 0))

(cmpr2 '(10 100 1000))
>>> 3
```

Outro exemplo: `inverte`

- ▶ Quando aplicado a listas, o processo iterativo naturalmente inverte a ordem das listas.
- ▶ Devido a isso, é trivial implementar a inversão de listas usando acumuladores.
- ▶ Em funções envolvendo listas em geral, os resultados ficam invertidos, o que é indesejado.
- ▶ **A solução é chamar `inverte` no caso base.**

```
(define (inverte l)
  (inverte-aux l '()))

(define (inverte-aux l acc)
  (if (null? l)
      acc
      (inverte-aux (cdr l) (cons (car l) acc))))
```

Revisitando velhos amigos

Faça versões iterativas para exercícios passados:

- ▶ soma
- ▶ contar
- ▶ mapear
- ▶ gaguinho
- ▶ alternar
- ▶ Etc. (escolha + 2)

Problemas que requerem acumuladores

- ▶ Há certos problemas que naturalmente requerem alguma lembrança do que já aconteceu em chamadas recursivas anteriores.
- ▶ Essa memória deve ser passada adiante através de acumuladores extras.
- ▶ `pos+` : Veja o caso do exercício abaixo, em que o resultado é o valor da lista original mais sua posição na lista.

```
(pos+ '(7 5 1 4))  
>>> '(7 6 3 7)
```

não-repete (ou *unique*)

- ▶ Elementos repetidos em sequência só contam uma vez.

```
(não-repete '(a a a b b a a a c c))  
>>> '(a b a c)
```

nunca-antes (ou *first-occurrence*)

- ▶ Registra elementos que aparecem pela primeira vez.
- ▶ Embora pareça com o anterior, existe uma solução sem acumulador.
- ▶ Dica: use `pertence?` (no Racket: `set-member?`)

```
(nunca-antes '(a a a b b a a a c c))  
>>> '(a b c)
```

Mergesort

- ▶ Faz o inverso do zíper.
- ▶ Retornar um par de listas.
- ▶ Dica: Use dois acumuladores.

```
(particionar '(1 2 a -3 0 7 c))  
>>> '((1 a 0 c) . (2 -3 7))
```

Mergesort

- ▶ Particione a lista recebida em duas, usando a função `particionar` anterior. Use `let` para criar variáveis locais.
- ▶ Recursivamente, ordene as duas sublistas. Use `let` para organizar seu código com variáveis locais.
- ▶ Use `misturar` para criar uma lista única ordenada a partir das duas sublistas ordenadas recursivamente no passo anterior.

reduzir

- ▶ Explicitamente recebe uma função um acumulador e uma lista.
- ▶ A função deve aceitar dois parâmetros, um referente a um acumulador, e outro referente a um elemento da lista.
- ▶ Muito usada, também conhecida como *reduce*, *fold_left*, *foldl* ou simplesmente *fold* (dobrar).
- ▶ Em Racket, é o `foldl`.
- ▶ (Qual a diferença do `foldr`?)

```
(reduzir '+ 0 '(1 2 3))  
>>> 6  
(reduzir max -1000 '(7 0 -3 8 5))  
>>> 8
```